Universidade Federal de Minas Gerais
Departamento de Ciência da Computação

Trabalho Prático 1 - Algoritmos I

Lívia Caroline Rodrigues Pereira
2024024003
liviacaroline456@gmail.com

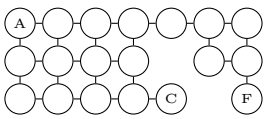
1 Introdução

O presente documento apresenta uma explicação e justificativa de código implementado como resolução do Trabalho Prático I da matéria Algoritmos I, cursada na Universidade Federal de Minas Gerais. O problema proposto consiste em determinar se um artista, partindo de uma posição inicial em um mapa retangular dividido em células, consegue alcançar um camarim antes de ser interceptado por um ou mais grupos de fãs que se espalham simultaneamente pelo local. O artista desloca-se com velocidade V células por segundo, enquanto os fãs avançam uma célula por segundo, ocupando permanentemente cada célula alcançada. O objetivo é modelar essa dinâmica como um problema de busca em grafos, determinando o menor número de movimentos necessário e a quantidade de caminhos distintos que realizam esse percurso mínimo. A solução proposta consiste na execução de duas Buscas em Largura (BFS) sucessivas, comparando o instante de chegada do artista com o instante de chegada dos fãs em cada célula do mapa e, posteriormente, a impressão do número mínimo de movimentos e da quantidade de caminhos mínimos (quando possível).

2 Modelagem

Todo o problema se baseia em um mapa de entrada, que define a estrutura dos corredores entre o Artista, os Fãs e o Camarim. Foi então estabelecido que o grafo seria o próprio mapa: as células que não são paredes, representando os corredores, são modelados como nós; e cada aresta representa corredores vizinhos. Assim, é modelado um grafo não direcionado $G = (N, E)$, com N sendo o conjunto de células que não são paredes (os corredores). Duas células u e v formam uma aresta $\{u, v\} \in E$ se, e somente se, compartilham um lado, isto é, se são adjacentes na horizontal ou na vertical. Movimentos diagonais não são permitidos, portanto não formam arestas. Para entender como o mapa se transforma em grafo, é apresentado a seguir um exemplo prático, com o formato de entrada e o grafo correspondente, respectivamente:

```
5 9 2
#########
#A......#
#....#..#
#....C#F#
#########
```



Nesse caso de teste, o mapa contém 5 linhas e 9 colunas, com o camarim ocupando a célula $(3, 5)$, o grupo de fãs iniciando em $(3, 7)$ e o artista em $(1, 1)$; o artista caminha com velocidade $V = 2$, o que significa que, para mudar de célula, ele gasta $1/2$ segundo, realizando dois movimentos a cada segundo. Simulando o funcionamento do sistema a ser implementado para um dos caminhos mínimos possíveis, temos que, ao final do instante $t = 1$ segundo, os fãs congestionam as células $(3, 7)$ e $(2, 7)$, enquanto o artista, tendo realizado dois movimentos, encontra-se na célula $(2, 2)$, após passar por $(1, 2)$. Ao final do instante $t = 2$ segundos, os fãs já congestionaram também as células $(1, 7)$ e $(2, 6)$, e o artista, tendo completado quatro movimentos, encontra-se na célula $(2, 4)$, após passar por $(2, 3)$.

Como cada célula possui no máximo quatro vizinhas (acima, abaixo, à esquerda e à direita), o grau de qualquer vértice de G é limitado por uma constante. Isso implica que o número de arestas cresce linearmente com o número de vértices, isto é, $|E| = O(|N|)$, caracterizando G como um grafo esparso.

Isso motiva a principal decisão de representação adotada: o grafo é armazenado de forma implícita, de forma que, em vez de construir uma lista de adjacência ou matriz de adjacência para representar as

arestas explicitamente, cada célula do mapa é identificada por um único índice inteiro, obtido a partir de suas coordenadas (i, j) pela fórmula

$$\text{idx} = i \cdot C + j,$$

onde C é o número de colunas do mapa. Os vizinhos de uma célula são obtidos somando ou subtraindo deslocamentos fixos a esse índice (± 1 para movimentos horizontais e $\pm C$ para movimentos verticais), com a devida verificação de limites do mapa e de bordas laterais.

3 Implementação

3.1 Instruções e Configurações Gerais

Em relação ao código entregue via arquivo compactado, a compilação deve ser feita pelo comando make, com a execução do programa sendo possível pelo comando ./ tp01 < casoTeste.txt. Importante ressaltar que o arquivo casoTeste.txt deve conter os valores que especificam o espaço dos eventos, sendo decodificado como matriz. A primeira linha do arquivo deve portanto conter, em ordem, número de linhas (L), número de colunas (C) e velocidade do artista (V). Importante ressaltar:

$$2 \leq L, C \leq 100 \qquad 1 \leq V \leq 5 \qquad L \cdot C \leq 10^4$$

E para cada célula (corredor) da matriz L x C, deve, nas próximas linhas, especificar se é corredor livre (.), parede (#), posição inicial do artista (A), posição inicial de um grupo de fãs (F) ou camarim (C).

Todo o projeto foi desenvolvido na linguagem C++, utilizando o editor Visual Studio Code, com compilação por meio do `g++`, da suíte GNU Compiler Collection, sob o padrão C++17. A estrutura do projeto é dividida em três arquivos principais: (i) `main.cpp` contém a execução principal, responsável pela leitura da entrada, exploração do mapa, chamada das duas buscas em largura e impressão do resultado final; (ii) `bfs.hpp`: contém as assinaturas das funções de busca e as constantes globais compartilhadas entre os arquivos; e (iii) `bfs.cpp`: contém a implementação das duas buscas em largura que resolvem o problema e função auxiliar de validação dos viznhos.

3.2 Estruturas de Dados e Algoritmos

Com o problema devidamente modelado (como mostra a Seção 2), a implementação surge de forma natural. Essa seção da documentação apresenta detalhes de escolhas algoritmicas durante o processo de codificação. Para o entendimento da explicação a seguir, além das nomenclaturas de entrada, definidas na Seção 3.1, é essencial salientar que `no` denota o vértice sendo processado pela busca naquele momento e `viz` denota um de seus vizinhos candidatos.

Primeiramente, o código recebe por entrada o formato especificado, lendo a partir do *cin* e percorrendo todas as células para encontrar A, C e F em um loop duplo (passando por cada célula do mapa), que testa os três caracteres de interesse em uma única cadeia de condicionais (`if`/`else if`). Enquanto as coordenadas de A e C são armazenadas em variáveis simples, as de F ficam nos vetores `linhaF` e `colunaF`, já que pode haver mais de um grupo de fãs no mesmo mapa.

Com o mapa devidamente explorado, o grafo é representado a partir de índices únicos que identificam cada célula: Dada uma célula de coordenadas (i, j), seu índice achatado é definido por $\text{idx} = i \cdot C + j$, e a operação inversa, que recupera a linha e a coluna a partir do índice, é dada por

$$i = \lfloor \text{idx}/C \rfloor, \qquad j = \text{idx}\%C.$$

com $j = idx\%C$ representando o resto da divisão inteira de idx por C. Essa representação permite que todas as estruturas auxiliares do algoritmo (`tFas`, `passos`, `caminhos`) sejam vetores unidimensionais de tamanho $L \cdot C$.

Ademais, a organização por índices facilita o cálculo dos vizinhos de uma célula, que pode ser feito diretamente: os vizinhos verticais são obtidos por $\text{idx} - C$ (célula acima) e $\text{idx} + C$ (célula abaixo), enquanto os vizinhos horizontais são obtidos por $\text{idx} - 1$ (esquerda) e $\text{idx} + 1$ (direita). É necessário, no entanto, tratar um caso particular: somar ou subtrair 1 ao índice de uma célula na última ou na primeira coluna de uma linha resultaria em um índice pertencente à linha adjacente, que pode não existir no mapa. Para evitar esse erro, os deslocamentos horizontais são condicionados à verificação da coluna da célula de origem, de modo que o vizinho à direita só é considerado válido se $j + 1 < C$, e o vizinho à esquerda só é considerado válido se $j - 1 \geq 0$.

Os dados calculados ficam em três vetores declarados em escopo global e repassados às buscas como parâmetros: `tFas` representa o instante em que o grupo de fãs mais próximo alcança cada célula; `passos` representa o número mínimo de movimentos que o artista precisa para alcançar cada célula; e `caminhos` representa a quantidade de caminhos mínimos distintos que levam a cada célula. Os arrays são definidos com tamanho constante (`MAXN`), já que o enunciado garante $L \cdot C \leq 10^4$, evitando alocação dinâmica desnecessária. É definida ainda a constante `INF`, utilizada para representar um tempo infinito nas células que nunca são alcançadas pelos fãs. Por fim, a constante `MOD` $= 10^9 + 7$ é definida para que a quantidade de caminhos seja informada módulo esse valor (como pede as instruções do trabalho).

As duas buscas que resolvem o problema compartilham a mesma estrutura geral de uma busca em largura: marcam todos os vértices como não visitados, inserem a origem (ou origens) na fila e, enquanto a fila não estiver vazia, retiram um vértice, examinam seus vizinhos e registram a informação correspondente para os que ainda não foram visitados. A diferença central entre elas está no número de origens: a busca dos fãs, `bfsFas`, é multi-fonte, iniciando simultaneamente a partir de todos os grupos de fãs presentes no mapa, com `tFas` igual a zero em cada uma dessas células; a busca do artista, `bfsArtista`, parte de uma única origem, a posição inicial do artista, com `passos` igual a zero e `caminhos` igual a um.

A busca do artista ainda acrescenta duas verificações que não existem na busca dos fãs: Primeiramente, um vizinho só é aceito se o artista conseguir alcançá-lo estritamente antes dos fãs, isto é, se `passos[no] + 1 < V*tFas[viz]`; a busca dos fãs, por não competir com nenhum outro agente, aceita qualquer vizinho ainda não visitado. Segundo, ao encontrar um vizinho já visitado, a busca do artista verifica se ele pertence ao mesmo nível de distância que seria alcançado agora (`passos[viz] == passos[no] + 1`); em caso positivo, soma-se `caminhos[no]` a `caminhos[viz]`, contabilizando um caminho mínimo adicional, sem reinserir o vértice na fila. A busca dos fãs não realiza essa contagem, já que seu único objetivo é determinar o instante de congestionamento de cada célula, e não o número de caminhos até ela.

A condição de segurança decorre dos instantes de chegada: o artista alcança o vizinho no instante (`passos[no]`$+1)/V$, e os fãs, no instante `tFas[viz]`. Como a chegada simultânea resulta em abordagem, o artista só pode acessar aquela célula se (`passos[no]` $+ 1) <$ `tFas[viz]` $\cdot V$. Como a fila retira todos os vértices de um nível antes dos do nível seguinte, o valor de `caminhos` de um vértice já está completo quando ele é processado.

Após a execução das duas buscas, o `main` consulta diretamente as posições `passos[idxCamarim]` e `caminhos[idxCamarim]`, onde `idxCamarim` é o índice achatado da célula do camarim. Se `passos[idxCamarim]` permanecer igual a -1, conclui-se que o camarim é inalcançável, e o programa imprime `-1 0`; caso contrário, imprime os dois valores encontrados, representando o número mínimo de movimentos e a quantidade de caminhos mínimos correspondentes (em módulo $(10^9 + 7)$), respectivamente.

4 Análise de Complexidade

A análise de complexidade da resolução proposta é guiada pelos valores das variáveis essenciais L (número de linhas do mapa) e C (número de colunas do mapa), sendo $n = L \cdot C$ o número total de células do mapa. Nesta seção, C denota exclusivamente o número de colunas, e não o caractere que marca o camarim no mapa.

4.1 Complexidade de tempo

O algoritmo utiliza o método de Busca em Largura (BFS), cuja complexidade genérica, para um grafo $G = (N, E)$ com $|N|$ vértices e $|E|$ arestas, é $O(|N| + |E|)$. Isso ocorre porque cada vértice é inserido na fila e processado no máximo uma vez, e cada aresta é percorrida no máximo duas vezes (uma a partir de cada extremidade), de modo que o trabalho total é proporcional à soma do número de vértices e arestas.

No grafo que modela o mapa, o número de vértices é limitado pelo número de células não-parede, ou seja, $|N| \leq L \cdot C = n$. Além disso, como cada célula possui no máximo quatro vizinhas (movimento restrito às direções ortogonais, sem diagonais), o grau de qualquer vértice é limitado por uma constante. Isso implica que

$$|E| \leq \frac{4 \cdot |N|}{2} = 2|N| \leq 2n,$$

onde a divisão por dois ocorre porque cada aresta é contabilizada a partir de suas duas extremidades. Substituindo esses limites na complexidade genérica da BFS, obtemos

$$O(|N| + |E|) = O(n + 2n) = O(3n) = O(n) = O(L \cdot C).$$

A solução proposta executa duas buscas em largura de forma sequencial: uma busca multi-fonte a partir dos grupos de fãs, responsável por calcular o instante de congestionamento de cada célula, e uma busca a partir da posição inicial do artista, responsável por calcular a distância mínima e a contagem de caminhos mínimos até o camarim. Como as duas buscas são executadas uma após a outra, seus custos se somam:

$$O(L \cdot C) + O(L \cdot C) = O(L \cdot C).$$

A busca multi-fonte é essencial para esse limite: executar uma busca independente a partir de cada um dos k grupos de fãs e tomar o mínimo custaria $O(k \cdot L \cdot C)$, podendo chegar a $O((L \cdot C)^2)$. Ao inserir todas as origens na fila com tempo zero, uma única busca calcula diretamente, para cada célula, o instante de chegada do grupo mais próximo.

Somando a isso o custo de leitura da entrada e de localização das posições iniciais do artista, do camarim e dos grupos de fãs, que também é $O(L \cdot C)$, a complexidade de tempo total do algoritmo é

$$\boxed{O(L \cdot C)}.$$

Como o enunciado garante $L \cdot C \leq 10^4$, o algoritmo realiza, no pior caso, uma quantidade de operações da ordem de dezenas de milhares. Esse limite é assintoticamente ótimo, já que a própria leitura do mapa exige $\Omega(L \cdot C)$ operações.

4.2 Complexidade de espaço

A complexidade de espaço é determinada pelas estruturas de dados utilizadas, todas dimensionadas em função de $n = L \cdot C$: (i) o mapa de entrada, armazenado como um vetor de L strings de C caracteres, ocupando $O(L \cdot C) = O(n)$; (ii) o vetor `tFas`, com uma posição por célula, $O(n)$; (iii) o vetor `passos` (distância do artista), $O(n)$; (iv) o vetor `caminhos` (contagem de caminhos mínimos), $O(n)$; (v) o vetor utilizado como fila, local a cada busca (de modo que apenas um existe por vez), com no máximo n posições, já que cada célula é inserida na fila no máximo uma vez, $O(n)$; e (vi) os vetores `linhaF` e `colunaF`, com as posições iniciais dos grupos de fãs, que no pior caso têm uma posição por célula, $O(n)$. Como todas essas estruturas têm tamanho proporcional a n e são em número constante, a soma delas permanece da mesma ordem:

$$O(n) + O(n) + O(n) + O(n) + O(n) + O(n) + O(n) = O(7n) = O(n).$$

Portanto, a complexidade de espaço total do algoritmo é

$$\boxed{O(L \cdot C)}.$$

4.3 Análise Experimental

Para validar empiricamente a análise teórica, foram gerados mapas quadrados aleatórios, sem paredes, com tamanhos variando de 100 a 10 000 células, com velocidade do artista fixa em $V = 3$. Para cada tamanho, o programa foi executado 15 vezes, e o menor tempo entre as repetições foi registrado, reduzindo o viés de velocidade do sistema operacional. O tempo foi medido internamente pelo próprio programa, por meio de `std::chrono`.

Como mostra a Figura 1, o tempo de execução cresce de forma aproximadamente proporcional ao tamanho do mapa: ao dobrar o número de células, de 1980 para 3969, o tempo também praticamente dobra, passando de 0,23 ms para 0,41 ms; o mesmo padrão se repete ao dobrar novamente, de 3969 para 7921 células, com o tempo indo de 0,41 ms para 0,74 ms. Essa proporcionalidade é exatamente o comportamento esperado de um algoritmo $O(L \cdot C)$: dobrar o tamanho do mapa dobra a quantidade de trabalho realizado. Já nos mapas menores, entre 100 e 500 células, o tempo por célula aparece levemente maior, o que pode ser atribuído a custos fixos de medição.

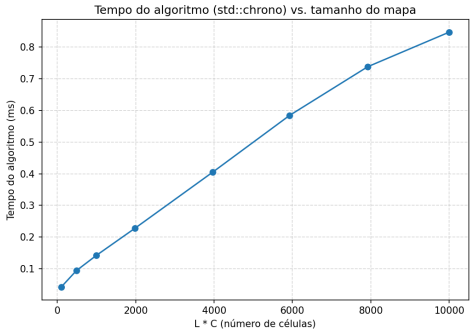


Figura 1: Tempo de execução do algoritmo em função do número de células do mapa ($L \cdot C$).

5 Conclusão

A realização deste trabalho objetivou consolidar a aplicação prática de algoritmos e estruturas de dados em grafos na resolução de um problema real de busca com restrições temporais, exercitando a modelagem de um mapa como grafo implícito, o uso de busca em largura e a análise de complexidade da solução proposta. Por fim, é importante salientar que, durante todo o processo de criação da resolução para o problema proposto, foi possível desenvolver novas formas de pensar e observar processos, exercitando uma lógica computacional pautada em compromissos necessários para a realização de determinadas ações.

6 Bibliografia

Cormen, Thomas H. Algoritmos: Teoria e Prática, Editora Campus, v. 2 p. 296, 2002 Ziviani, Nívio. Projeto de Algoritmos. Cengage Learning, 2006.